ACTIVITAT
Objectius: ■ Processat de dades amb JSON en MongoDB
Instruccions: ■ Resoleu els exercicis proposats i pengeu el repositori a GitHub
Criteris d'avaluació: ■ Es valorarà la presentació i els comentaris al codi
Entrega: ■ A Moodle enllaç a repositori GitHub. - A dins del repositori, en un directori anomenat "doc" inclout aquest document completat en format pdf i amb el nom "memoria.pdf"
Repositori d'exemple: ■ https://github.com/jpala4-ieti/DAM-M0486-Tema4-RA5-PR3.2-Punt-Partida-25-26

Exercicis

Exercici 1. Mini-Servidor WebSocket de Registre d'Esdeveniments de Joc amb Client Node.js (10 punts)

Crear un servidor WebSocket bàsic en Node.js que escolti connexions entrants. El servidor rebrà missatges en format JSON simulant el moviment 2D d'un jugador en una partida des d'un client Node.js i els emmagatzemarà en una col·lecció de MongoDB.

Tecnologies a utilitzar:

- Node.js
- **Biblioteca ws:** Per implementar WebSockets tant al servidor com al client (npm install ws).
- MongoDB (utilitzant la mateixa instància de Docker que a la PR3.1)
- Driver de MongoDB (mongodb).

Què cal fer?

- Crea una fitxa de requisits per la funcionalitat que es demana resoldre. Parteix de la informació de l'annex i amb ajuda de la IA Claude, indica quins altres aspectes poden millorar la fitxa de requisits **(2.5 punts)**
- Crea una carpeta anomenada pr32_websocket_link
- Crea dos fitxers: websocket_server.js i websocket_client.js cal implementar aquesta funcionalitat **(5.5 punts)**:
    - El jugador s'ha de moure amb les fletxes.
    - El client enviarà la informació al servidor.
    - Sempre i quan el jugador estigui en moviment la informació de posició es considerarà la mateixa partida. Cada moviment en guardarà a MongoDB (1 document per moviment, però amb informació per associar-lo a la partida).
- Si el jugador passa 10 segons sense moure's es considera finalitzada la partida, el calcula la distància en línia recta entre el punt inicial i final i s'informa al client **(2 punts)**
- En els servidor cal implementar logs (pantalla i fitxer) amb winston.

Mini-Servidor WebSocket de Registre d'Esdeveniments de Joc amb Client Node.js

1. Objectiu principal

Desenvolupar un sistema client-servidor utilitzant WebSockets i Node.js per capturar i emmagatzemar el moviment 2D d'un jugador en temps real. Les dades s'han de persistir

en una base de dades MongoDB, identificant sessions de joc (partides) i calculant la distància entre el primer i últim punt de moviment. La partida finalitza quan no s'hi rep cap input de l'usuari durant 10 segons.

2. Requisits funcionals

- ***RF-01*** (Connexió): El servidor ha d'establir connexions WebSocket amb el/s client/s.
- **RF-02** (Captura de Moviment): El client ha de capturar les pulsacions de les tecles de fletxa per moure el jugador.
- **RF-03** (Transmissió de Dades): El client ha d'enviar en coordenades (dx,dy) quina tecla de moviment ha sigut pulsada:
  - +1 - Adalt/Dreta.
  - -1 - Abaix/Esquerra.
  - 0 - Sense moviment.
- **RF-04** (Gestió de Partida): El servidor ha d'agrupar els moviments sota un mateix identificador de partida mentre el jugador estigui actiu. Aquest es genera quan el client es connecta i genera una de nova quan partida finalitza.
- **RF-05** (Persistència): Cada moviment rebut s'ha de guardar com un document individual a MongoDB, incloent l'ID de la partida.
- **RF-06** (Detecció d'Inactivitat): Si el servidor no rep moviments del client durant 10 segons, s'ha de considerar la partida com a finalitzada.
- **RF-07** (Càlcul de Resultats): En finalitzar la partida, el servidor ha de calcular la distància en línia recta entre el punt inicial (primer moviment) i el final (últim moviment).
- **RF-08** (Notificació): El servidor ha d'informar el client de la distància total recorreguda un cop tancada la partida.

3. Requisits no funcionals

- **RNF-01** (Logging): S'ha d'utilitzar la biblioteca Winston per generar logs tant a la consola com a un fitxer de text.
- **RNF-02** (Tecnologies): L'aplicació s'ha de basar en Node.js, el driver oficial de MongoDB i la llibreria ws.
- **RNF-03** (Infraestructura): MongoDB s'ha d'executar en un contenidor Docker.
- **RNF-04** (Mantenibilitat): El codi ha d'estar comentat segons els criteris d'avaluació.

4. Format del missatge JSON

Client > Servidor

```
{
 "dx": -1 | 0 | 1,
 "dy": -1 | 0 | 1
}
```

Servidor > Client

```
{
 "type": "ack",
 "moveNumber": 1,
 "position": {
   "x": 3,
   "y": -2
 }
}
```

```
{
 "type": "game_over",
 "sessionId": "4773e605-84ba-41e0-82b6-954056533b56",
 "totalMoves": 45,
 "startPosition": { "x": 0, "y": 1 },
 "endPosition": { "x": -3, "y": 6 },
 "straightLineDistance": 5.83
}
```

Millores proposades per la IA Claude Haiku 4.5

"He revisat la fitxa de requisits del Sistema de Monitorització de Sensors. Aquí et proposo millores estratègiques i específiques:

🔴 MILLORES CRÍTICS

1. **Seguretat (FALTA COMPLETAMENT)**

- Afegir RF-13: Autenticació de clients (token, API key, JWT)
- Afegir RF-14: Validació de rangs numèrics dins del JSON (ex: value no pot ser NaN, Infinity)
- Afegir RNF-04: Rate limiting per client (ex: màx 100 lectures/minut per evitar DoS)
- Afegir RNF-05: Validació de format sensorId (ex: permet només alfanumèric + guió)

2. **Gestió de Dades (INCOMPLET)**

- Afegir RF-15: Crear índexs a MongoDB: sensorId, type, timestamp per optimitzar queries
- Afegir RNF-06: Política de retencció (ex: borrar dades > 90 dies automàticament)
- Afegir RF-16: Impedir duplicats: dues lectures idèntiques en < 1 segon = discartar

3. **Monitorització (VAGUE)**
- Millorar RNF-01: Definir nivells de log amb stack traces per errors
- Afegir RF-17: Endpoints de métriques (ex: /metrics amb Prometheus)
- Afegir RF-18: Dashboard de visualització (ex: Grafana per gràfics d'anomalies)

🟡 MILLORES IMPORTANTS

4. **Robustez i Recuperació**
- Ampliar RNF-03: Reconnexió automàtica de clients (backoff exponencial)
- Afegir RF-19: Queue de lectures (si BD falla, buffer a memòria fins a 10K documents)
- Afegir RNF-07: Health checks periòdics servidor ↔ DB

5. **Documentació i API**
- Afegir RF-20: Documentació OpenAPI/Swagger del format JSON
- Afegir RF-21: Exemple de codi client complet amb manualitat d'errors
- Afegir RNF-08: Changelog de versions i versionat de API

6. **Testing i Qualitat**
- Afegir RF-22: Suite de tests unitaris (mínim 80% cobertura)
- Afegir RF-23: Test d'integració: enviar 1000 lectures/s i verificar BD
- Afegir RNF-09: Linter (ESLint) i prettier per format de codi

🟢 MILLORES D'ESCALABILITAT

7. **Escalabilitat Futura**
- Afegir RNF-10: Preparar per clustering (Node.js cluster module)
- Afegir RNF-11: Documentar com deployar amb Docker/K8s
- Afegir RNF-12: Message broker opcional (Redis/RabbitMQ) per a multi-servidor

8. **Detecció d'Anomalies Mejorada**
- Ampliar RF-11: No només por-de-rang, sinó detectar:
- Salts bruscos de valor (diff > 10°C en 10s)
- Lectures repetides igual (possible sensor fes)
- Freqüència anòmala (massa sovint o massa tard)

📝 EXEMPLE FITXA MILLORADA (Fragment)

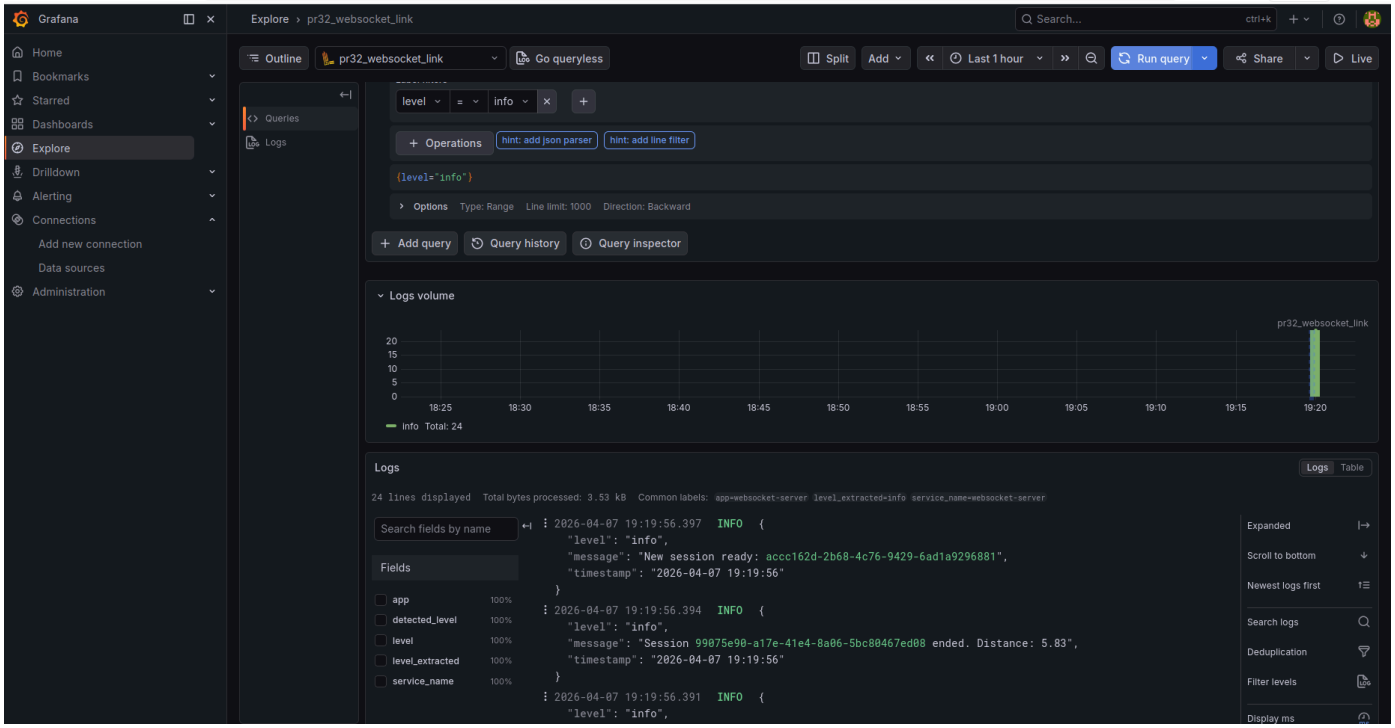
RF-13: Els clients REQUEREIXEN autenticació via API Key (header: X-API-Key)
RF-14: El servidor REBUTJA lectures on value no sigui number real entre -999 i +999
RF-15: MongoDB TINDRÀ índexs compostos: {sensorId, type, timestamp}
RNF-04: Rate limiting: màx 100 lectures/minut per sensorId (sinó: desconnexió)
RNF-06: Retencció: eliminar automàticament documents > 90 dies menstualment
RF-16: Deduplicació: si (sensorId, type, value) = lectura anterior < 1s: discartar

💡 MÓN CONTEXT

La fitxa actual és sòlida per a MVP però li falten aspectes de seguretat, monitorització, i escalabilitat crítics per producció. Vols que reformuli tota la fitxa amb aquestes millores integrades, o que implementi algun d'aquests punts al codi?"

Ampliació d'Infraestructura (Grafana Loki)

1. L'entorn s'ha d'ampliar amb Grafana Loki (que actuarà com a base de dades especialitzada en logs) i Grafana (que proporcionarà la interfície visual per fer cerques i panells).
2. **Integració Node.js / Winston**: S'ha de modificar la configuració del servidor WebSocket per enviar les dades a Loki. Això s'implementarà afegint un nou *transport* directe a winston (ex: utilitzant la llibreria `winston-loki`) o configurant un contenidor agent (Promtail) que llegeixi l'arxiu físic de logs i l'enviï a Loki.



Annex

Exemple fitxa de requisits

Sistema de Monitorització de Sensors en Temps Real:

Què ens demanen: Crea un servidor Node.js que utilitzi WebSockets per rebre lectures simulades de sensors (per exemple, temperatura i humitat) en format JSON des de múltiples clients Node.js. Emmagatzema cada lectura rebuda, juntament amb un timestamp, en una col·lecció de MongoDB. El servidor podria implementar una lògica per detectar lectures anòmales (fora d'un rang esperat) i notificar-ho als clients o registrar-ho de manera especial.

Fitxa de Requisits: Sistema de Monitorització de Sensors

Autor: [El Teu Nom o Equip] **Data:** [Data de Creació, ex: 2025-03-27]

1. Objectiu Principal

Desenvolupar un prototip funcional amb Node.js que actuï com a servidor WebSocket per rebre lectures simulades de diversos sensors (com temperatura i humitat) des de clients Node.js. Les lectures rebudes s'han d'emmagatzemar en una base de dades MongoDB juntament amb un timestamp, i el sistema ha de poder identificar i registrar lectures que es considerin anòmales (fora d'un rang predefinit).

2. Requisits Funcionals (Què ha de fer)

- **RF-01:** El servidor WebSocket ha d'iniciar-se i escoltar connexions entrants en un port configurable
- **RF-02:** Els clients WebSocket (simulats) han de poder connectar-se al servidor.
- **RF-03:** Els clients han d'enviar lectures de sensors periòdicament en format JSON.
- **RF-04:** El format JSON de les lectures ha d'incloure `sensorId` (string), `type` (string, ex: "temperature"), i `value` (number).
- **RF-05:** El servidor ha de rebre i processar correctament els missatges JSON dels clients.
- **RF-06:** El servidor ha de validar l'estructura bàsica dels missatges rebuts. Els invàlids s'han de descartar i registrar (log).
- **RF-07:** El servidor ha d'afegir un `timestamp` (data i hora UTC) a cada lectura vàlida rebuda.
- **RF-08:** El servidor ha d'emmagatzemar cada lectura vàlida (amb `sensorId`, `type`, `value`, `timestamp`) com un document individual en una col·lecció de MongoDB.
- **RF-09:** El sistema ha de permetre configurar rangs de valors considerats normals per a cada tipus de sensor (ex: Temperatura 0-50°C).

- **RF-10:** El servidor ha de comprovar si el `value` de cada lectura rebuda cau dins del rang normal definit per al seu `type`.
- **RF-11:** El servidor ha de registrar de forma específica (log) qualsevol lectura detectada com a anomalia (fora de rang).
- **RF-12:** El servidor ha de gestionar les desconnexions dels clients i registrar l'esdeveniment.

3. Requisits No Funcionals (Com ho ha de fer)

- **RNF-01 (Logging):** El servidor ha d'implementar logs amb `winston`. S'han de registrar els esdeveniments clau (inici, connexió/desconnexió, error, dada rebuda, dada emmagatzemada, anomalia detectada) a la consola i a un fitxer (`sensor_server.log`).
- **RNF-02 (Mantenibilitat):** El codi ha d'estar comentat. La configuració essencial (port del servidor, URI de connexió a MongoDB, rangs d'anomalia) ha de ser fàcilment modificable (p. ex., variables d'entorn o fitxer de configuració).
- **RNF-03 (Fiabilitat Bàsica):** El servidor hauria de poder gestionar errors comuns (ex: error de connexió a BD) registrant l'error sense aturar-se completament, si és possible.

4. Format Missatge JSON (Client -> Servidor)

Els clients enviaran lectures amb l'estructura següent:

```
{
  "sensorId": "string", // Identificador únic del sensor (ex: "TEMP-001", "HUM-A")
  "type": "string",     // Tipus de mesura (ex: "temperature", "humidity")
  "value": number       // Valor numèric de la mesura (ex: 23.5, 45.2)
}
```